Lab 5

Designing a Pipelined RISC Processor (Part 2)

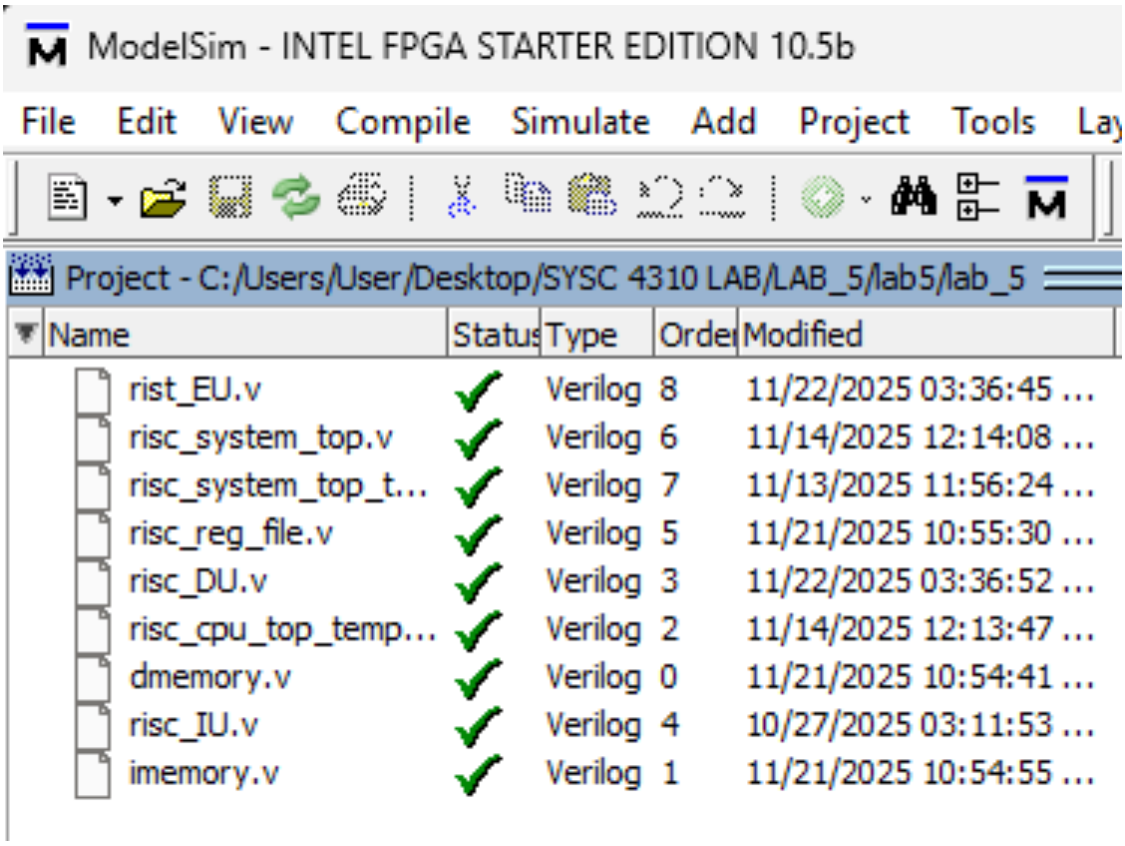
NAME: JABER-UL HUDA

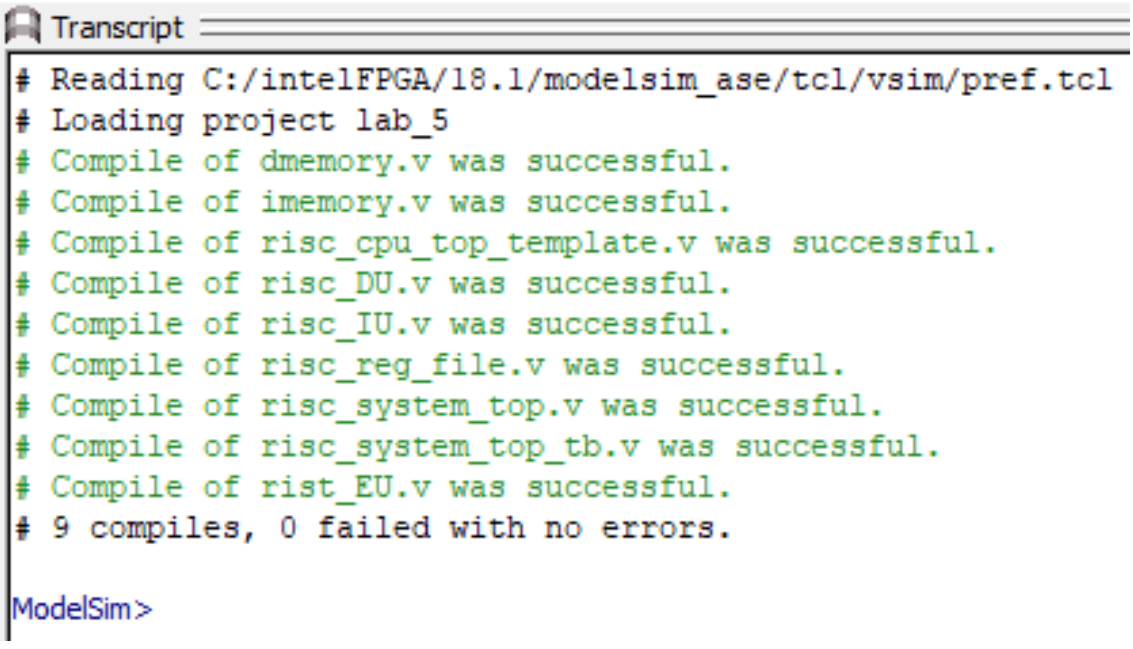
Student ID: 101137524

I. Introduction

In this lab, I completed the design of a 4-stage pipelined RISC processor by integrating the functional units I developed in the previous lab—namely the Instruction Unit (IU), Decode Unit (DU), Execution Unit (EU), and Register File—into a fully operational processor system. Building on my earlier simulations of individual components, this experiment focused on connecting the instruction memory and data memory to the CPU, structurally modeling the entire processor, and running a hand-assembled test program to verify correct instruction execution. Through this work, I gained practical experience in designing a pipelined datapath, routing signals between modules, interpreting opcode and operand flow, and validating processor behavior using ModelSim waveforms. This lab helped reinforce my understanding of how instructions move through each pipeline stage and how memory interactions support load, store, arithmetic, logical, and shift operations in a simple RISC architecture.

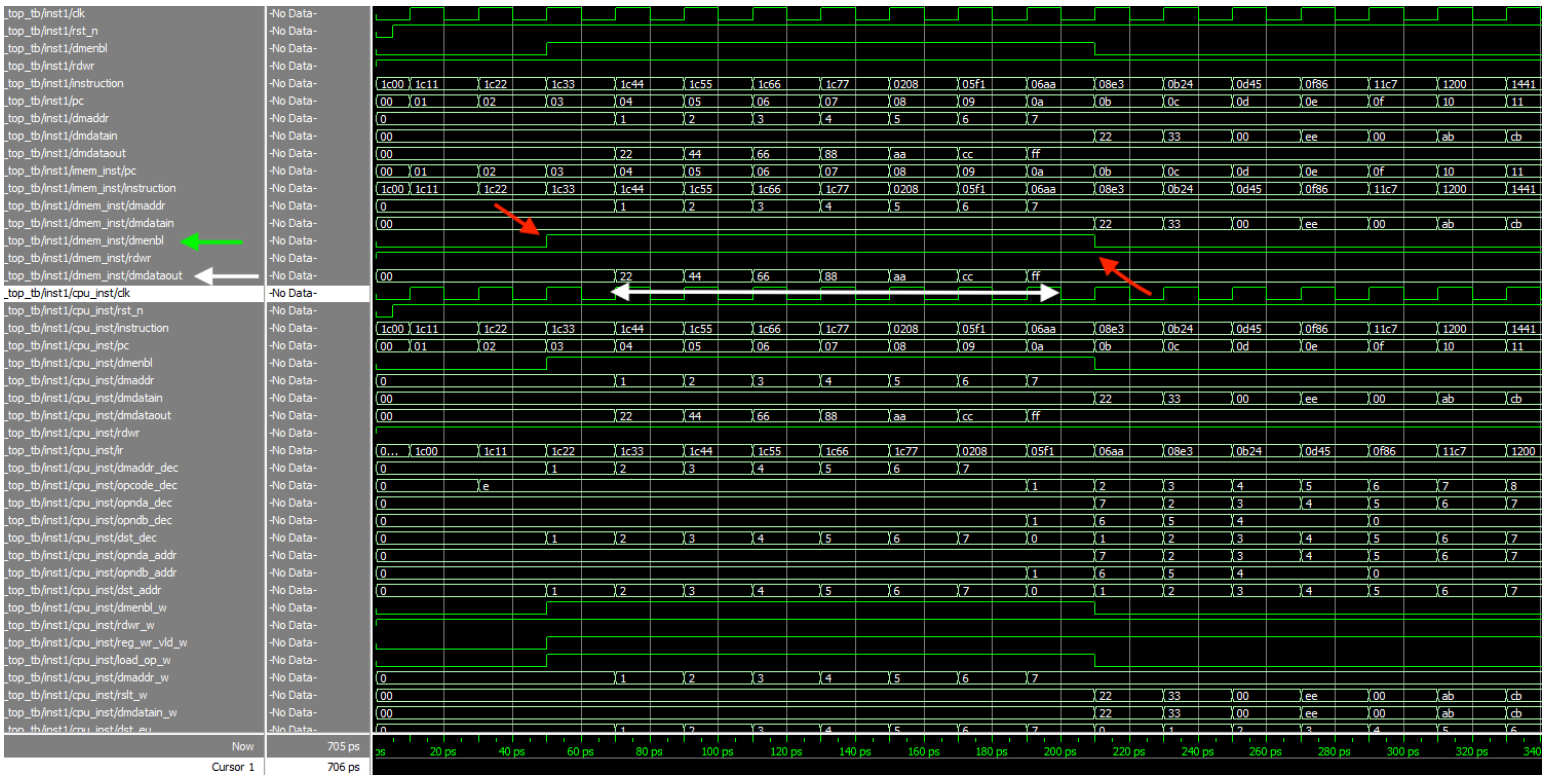
II. **Design (Task 1):** Take a screenshot of successful compilation of the CPU system.



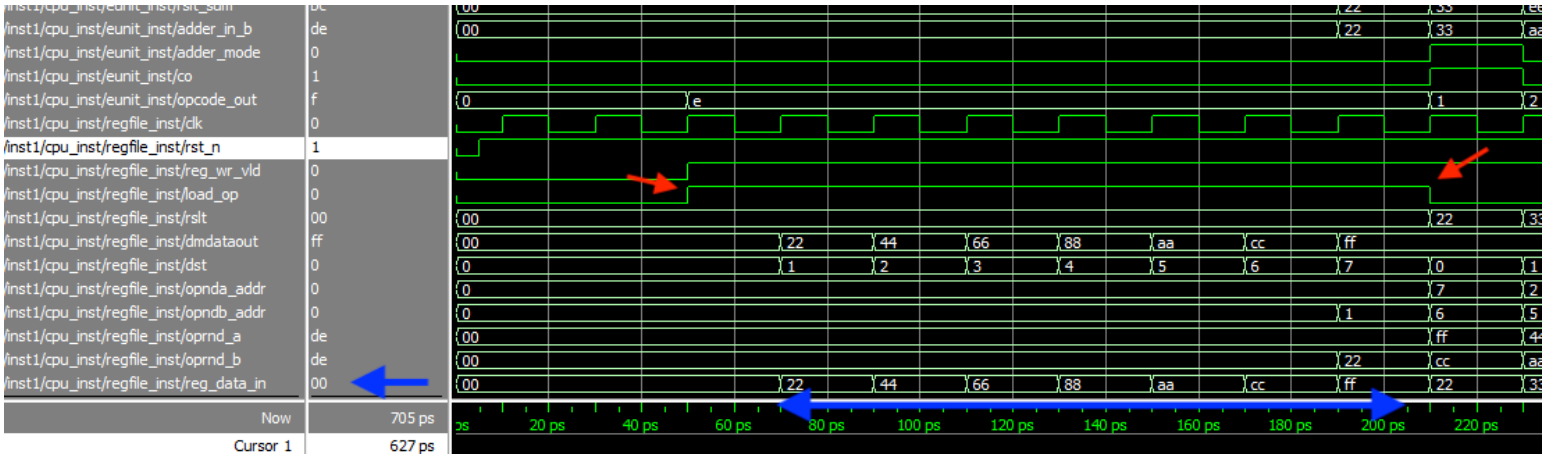


III. **Test (Task 2):** Include screenshots of the content of registers after all loads, and after the execution of the arithmetic and logical instructions.

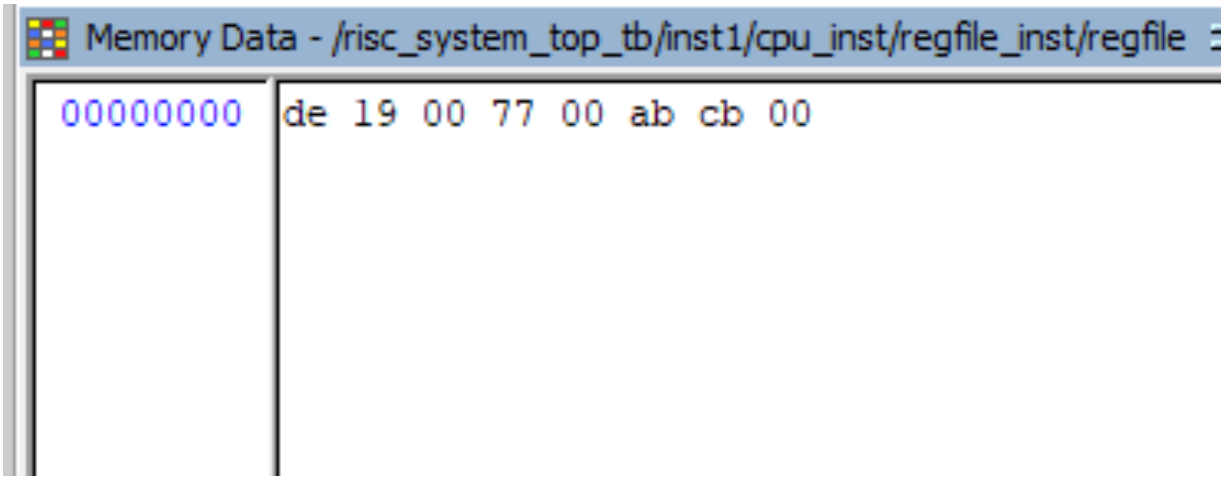
The wave graph shows the load portion of the instructions where r0 to r7 registers are loaded with data from memory. The white double arrow shows contents of the main memory being transferred to the registers using the dmdataout bus. The white single arrow shows the bus dmdataout. The green arrow shows the dmenbl set the high and the red arrows shows the duration it is high which is the time taken to move all the data from the main memory to the r0 to r7 registors.



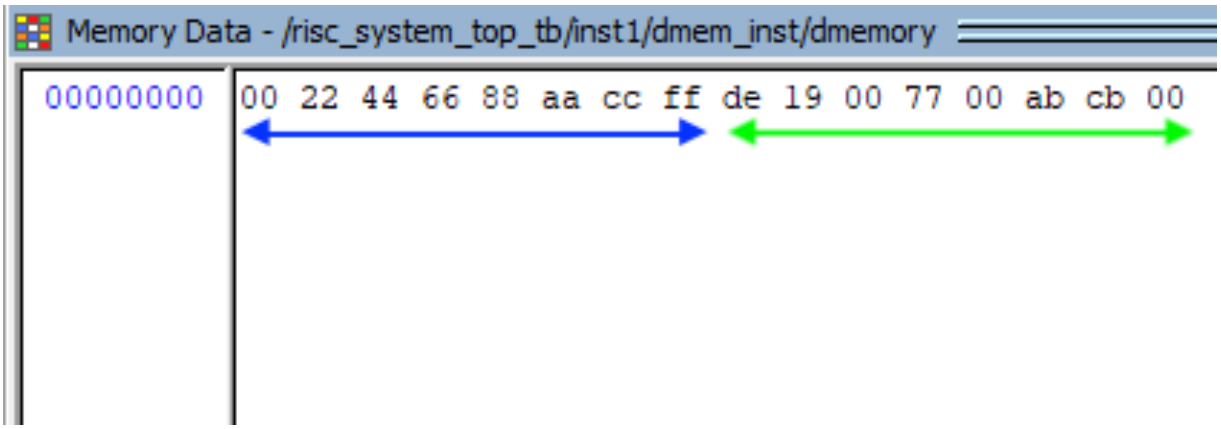
The below waveform shows the data getting inputted to the registers. The blue double arrow shows the data getting stored to the r0 to r7 registers using the reg_data_in bus. The red arrow shows the duration the load_op is set to high, and the duration is the time taken wo load all the memory location form r0 to r7 using data from the main memory. The dst address bus shows the address of the registers to which the data is being loaded to.



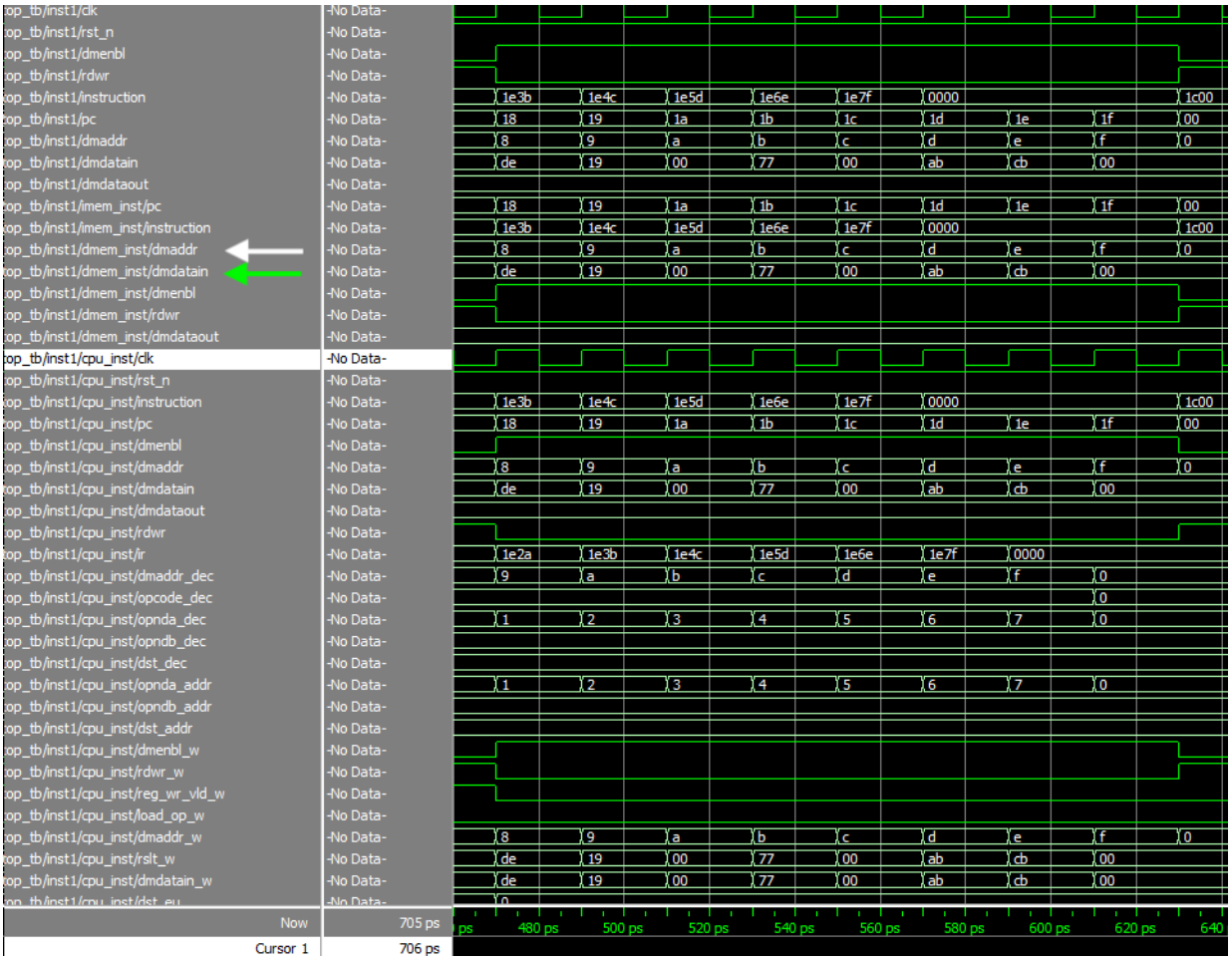
After all the load operations are done. The rest of the code is used to perform ALU operations on the data stored in the register file. The results of the operations are stored on the registers before storing them back to main memory using the store operations. The picture below shows the state of the register after all the ALU operations are completed. The data stored in r0 is the left most value 0xde. The data stored in r7 is the right most value 0x00.



The results from the registers are then loaded back into the main memory. Note that none of the initial data form the main memory is overwritten. All the contents from r0 to r7 after the ALU operations are loaded in new memory locations in the main memory. The blue double arrow shows all the data loaded to the registers before the ALU operations. The green double arrow shows all the data stored from the registers to the main memory after the ALU operations are done.



The below wave form shows the data that is loaded into the main memory. The green arrow shows the data getting loaded through the dmdatain data bus. The address in the main memory to which the data is getting loaded to is shown by the white arrow. It shows the dmaddr address bus.



IV. Conclusion

This lab allowed me to bring together all the core building blocks of a pipelined RISC processor and integrate them into a complete, functioning system. By connecting the instruction memory, data memory, and CPU modules, I gained a deeper understanding of how each stage of the pipeline interacts—especially how decoded operands, ALU results, and memory access signals flow across the design. Running the provided hand-assembled program reinforced my knowledge of instruction sequencing and helped me visualize processor behavior through waveform analysis in ModelSim. I also became more comfortable debugging structural connections, tracing register updates, and confirming correct load, store, arithmetic, logical, and shift operations. Overall, this experiment strengthened my understanding of pipelined architectures and gave me hands-on experience building and validating a synchronous digital processor from individual Verilog components.

V. **Copy the rubric (given on the next pages of this lab manual) and include it after the conclusion section in your lab report so that your teaching assistant can mark your report accordingly**.

Criteria	Level 4	Level 3	Level 2	Level 1	Criterion Score
Task 1: CPU top Verilog code	10 points the functional units are connected correctly in the top-level cpu verilog code	7 points The Verilog code for the top-level cpu has minor issues	4 points Incomplete or incorrect implementation	0 points this task is not completed.	/ 10
Task 2: Structural modelling of the whole system (Task 2 Verilog code)	10 points the top level system block has been connected correctly	7 points The Verilog code for the top-level system has minor issues	4 points The Verilog code for the top-level cpu has minor issues	0 points this block is not completed.	/ 10
Testbench and simulation	40 points The simulation is complete. It includes screenshots that shows various operation of the processor **(fetch, decode, execution, reading data from the memory, writing into memory).** It includes **separate screenshots for each of the above for clear demonstrations.** It shows the execution of all instructions. All instructions are executed correctly. Waveforms are annotated and explained.	30 points Simulations are included but have minor errors/ explanation is not included. Waveforms are not that clear, and does not show various operations of the processor.	20 points Simulations are included but it has major errors. For example, some signals might be red. Instructions may not be correctly executed.	0 points this task is not completed.	/ 40
Lab report and documentation	10 points All required screenshots are included and marked when necessary. The work is explained. Student name and ID are included in all codes.	7 points Lab report has minor issues (more explanation is needed, screenshots are not visible enough, organization is not that good)	4 points Lab report misses key sections and criteria outlined in level 4. The work is not explained well.	0 points The lab report does not meet the requirements. Most screenshots are missing, no explanations, etc.	/ 10
Checking out from the lab and answering questions	10 points The student attended the lab and explained his work thoroughly.	7 points The student could not answer some questions	4 points The student could not explain most of their work	0 points The student could not explain their work at all.	/ 10